

Guia 1 — Git Básico + Configurações

Este guia apresenta Git do zero, com explicações diretas, exemplos reais e saídas de terminal comentadas.

Ele foi pensado para quem está começando e quer entender o porquê de cada comando, não apenas decorar instruções.

Nota: Este guia cobre o fluxo essencial do Git.

Comandos como `git add` e `git commit` aparecem aqui em sua forma básica, pois fazem parte do funcionamento mínimo do Git.

Para técnicas avançadas, como adicionar trechos específicos, revisar diferenças e editar commits, consulte o Guia 2 (Staging Area e Commits Avançados).

1. O que é Git (em poucas palavras)

Git é um sistema de controle de versão distribuído.

Ele registra o histórico completo do seu projeto, permitindo comparar versões, voltar no tempo e trabalhar com segurança — tudo isso sem depender de internet.

Quando você ativa o Git em uma pasta, ele cria um diretório oculto chamado:

```
.git
```

Esse diretório transforma a pasta comum em um repositório Git, armazenando:

- histórico de commits
- referências internas
- branches
- metadados
- objetos compactados
- configurações específicas do repositório

Nunca altere manualmente a pasta `.git`:

- Ela é o “núcleo” do repositório, qualquer modificação incorreta pode corromper o histórico inteiro.

O Git funciona registrando snapshots (fotos) do estado do projeto a cada commit.

Ele não salva cópias completas dos arquivos repetidamente, apenas as diferenças, o que o torna extremamente rápido e eficiente.

Analogia:

Imagine o Git como um álbum de fotos do seu projeto.

Cada commit é uma nova foto, mostrando como tudo estava naquele momento.

📁 2. Iniciando um repositório — `git init`

Para começar a versionar uma pasta com Git, o primeiro passo é inicializar um repositório:

```
git init
```

Saída típica:

```
Initialized empty Git repository in C:/Users/ander/exemplo/.git/
```

Isso significa que:

- o Git criou um repositório vazio
- a pasta oculta `.git` foi criada e agora guarda todo o histórico e metadados do projeto

Você pode confirmar com:

```
ls -a
```

Saída típica:

```
.  ..  .git  arquivo1.py
```

A partir desse momento, a pasta deixa de ser comum e passa a ser um repositório Git.

📌 2.1. Quando usar `git init`?

Use `git init` quando:

- você está criando um projeto do zero
- deseja transformar uma pasta existente em um repositório Git
- quer começar a registrar histórico localmente, mesmo sem GitHub ou internet

🚫 2.2. Quando não usar `git init`?

Não use `git init` quando você está clonando um repositório:

```
git clone https://github.com/usuario/projeto.git
```

Nesse caso, o Git já cria a pasta `.git` automaticamente, então rodar `git init` dentro dela causaria conflitos.

⚙️ 3. Configurações do Git — system, global e local

O Git permite configurar seu comportamento em três níveis diferentes. Cada nível corresponde a um arquivo distinto e afeta um escopo específico:

Nível	Escopo	Arquivo
system	todos os usuários da máquina	<code>/etc/gitconfig</code>
global	apenas o usuário atual	<code>~/.gitconfig</code>
local	apenas o repositório atual	<code>.git/config</code>

✓ Ordem de prioridade

```
local > global > system
```

Se uma configuração existir no nível local, ela sobrescreve a mesma configuração nos níveis global e system.

📄 3.1. Ver configurações

Ver todas as configurações ativas

```
git config --list
```

Ver apenas as configurações globais

```
git config --global --list
```

Ver apenas as configurações locais

```
git config --local --list
```

Ver a origem de cada configuração

```
git config --show-origin --list
```

Exemplo de saída:

```
file:C:/Users/ander/.gitconfig  user.name=Anderson
file:.git/config                user.email=local@exemplo.com
```

3.2. Definir nome e e-mail

O Git precisa saber quem está criando os commits.

Essas informações aparecem no histórico e ajudam a identificar autores.

Configuração global (recomendada)

```
git config --global user.name "Seu Nome"
git config --global user.email "seu-email@exemplo.com"
```

Configuração local (somente para o repositório atual)

```
git config user.name "Nome Local"
git config user.email "email-local@exemplo.com"
```

Ver a origem das configurações

```
git config --show-origin --list
```

3.3. Remover (unset) configurações

Remover configuração local

```
git config --unset user.name
git config --unset user.email
```

Remover configuração global

```
git config --global --unset user.name
git config --global --unset user.email
```

Remover configuração system (raro e exige permissões elevadas)

```
git config --system --unset user.name
git config --system --unset user.email
```

✿ 3.4. Outras configurações úteis

Definir main como branch padrão

```
git config --global init.defaultBranch main
```

Definir editor padrão (VS Code)

```
git config --global core.editor "code --wait"
```

Ativar cores no terminal

```
git config --global color.ui auto
```

Definir `.gitignore` global

```
git config --global core.excludesfile ~/.gitignore_global
```

✦ Observação importante sobre `git config` e `git init`

Você pode configurar o Git antes de iniciar qualquer repositório.

Isso funciona porque:

- `git config --global` escreve no arquivo `~/.gitconfig`
- esse arquivo não depende de existir um repositório
- as configurações globais valem para todos os repositórios futuros

Ou seja:

Configurações globais não exigem que você tenha rodado `git init`.

🔍 4. Ver o estado do repositório — `git status`

O comando `git status` mostra a situação atual do repositório: quais arquivos estão sendo rastreados, quais foram modificados e o que está pronto para commit.

```
git status
```

Exemplo de saída:

```
On branch main
No commits yet
Untracked files:
  script.py
```

Interpretação:

- você está na branch `main`
- ainda não existem commits
- `script.py` não está sendo rastreado pelo Git

Outros estados importantes exibidos pelo `git status`:

- `Changes not staged for commit` → arquivo foi modificado, mas ainda não está na staging area
- `Changes to be committed` → arquivo já está na staging area e será incluído no próximo commit
- `working tree clean` → não há nada para adicionar, modificar ou commitar

O `git status` é um dos comandos mais importantes do fluxo básico, pois mostra exatamente o que o Git sabe (ou não sabe) sobre seus arquivos. Ele será usado constantemente ao longo deste guia e também no [Guia 2](#), mas aqui você aprende apenas o uso essencial.

5. Adicionar arquivos — `git add`

O comando `git add` coloca arquivos na *staging area*, preparando-os para serem incluídos no próximo commit.

Adicionar um arquivo específico:

```
git add script.py
```

Adicionar todos os arquivos modificados ou novos:

```
git add .
```

Confirmando o estado:

```
git status
```

Exemplo de saída:

```
Changes to be committed:  
  new file:   script.py
```

A staging area funciona como uma bandeja de supermercado:

- você coloca itens nela (`git add`)
- e depois confirma tudo de uma vez no caixa (`git commit`)

Nota: Aqui você aprende apenas o uso básico de `git add`, suficiente para o fluxo essencial do Git.

Técnicas avançadas, como, por exemplo, adicionar apenas partes de um arquivo (`git add -p`), são tratadas no [Guia 2](#), para evitar redundância entre os materiais.

6. Salvar uma versão — `git commit`

O comando `git commit` registra uma nova versão do projeto, criando um ponto no histórico que você pode consultar ou restaurar no futuro.

Criar um commit com uma mensagem clara:

```
git commit -m "Mensagem clara"
```

Exemplo de saída:

```
[main (root-commit) 1a2b3c4] Adiciona script inicial  
 1 file changed, 10 insertions(+)  
 create mode 100644 script.py
```

Interpretação:

- `root-commit` → é o primeiro commit do repositório
- `1a2b3c4` → hash único que identifica o commit
- `10 insertions` → 10 linhas foram adicionadas ao projeto

Boas práticas para mensagens de commit

- use mensagens claras e objetivas
- prefira verbos no imperativo (“Adiciona”, “Corrige”, “Remove”)
- descreva o que foi feito, não o porquê (o “porquê” pertence ao pull request ou issue)

Nota: Aqui você aprende apenas o uso essencial de `git commit`.

Técnicas avançadas, como editar o último commit (`--amend`) ou criar commits estruturados, são abordadas no [Guia 2](#), para manter este guia focado no fluxo básico.

7. `.gitignore` — ignorando arquivos corretamente

O arquivo `.gitignore` informa ao Git quais arquivos e pastas devem ser ignorados, ou seja, não rastreados, não adicionados, não commitados e não enviados ao repositório remoto.

Ele é essencial para manter o repositório limpo, seguro e leve.

7.1. Por que usar um `.gitignore`?

Você deve ignorar arquivos que:

- não fazem parte do código-fonte
- são gerados automaticamente
- contêm informações sensíveis
- são específicos da sua máquina
- não devem ir para o GitHub

Exemplos comuns:

- pastas de cache (`__pycache__/, .pytest_cache/`)
- ambientes virtuais (`venv/`)
- arquivos de configuração local (`.env`)
- logs (`*.log`)
- arquivos temporários (`*.tmp`)
- dependências reinstaláveis (`node_modules/`)

7.2. Onde criar o `.gitignore`?

Crie o arquivo na raiz do repositório, no mesmo nível da pasta `.git`:

```
touch .gitignore
```

Estrutura típica:

```
meu-projeto/  
├── .git/  
├── .gitignore  
├── src/  
└── README.md
```

7.3. O que escrever dentro do `.gitignore`?

Você adiciona padrões de arquivos ou pastas que o Git deve ignorar.

Exemplo básico:

```
__pycache__/  
.env  
*.log
```

Explicando:

- `__pycache__/` → ignora a pasta inteira
- `.env` → ignora o arquivo `.env`
- `*.log` → ignora todos os arquivos `.log`



7.4. Ignorando arquivos que já estão sendo rastreados

Se um arquivo já foi commitado antes de entrar no `.gitignore`, ele continuará sendo rastreado.

Para removê-lo do histórico sem apagar do disco:

```
git rm --cached arquivo.txt
```

Depois disso, o Git passa a ignorá-lo normalmente.



7.5. `.gitignore` global (para todos os repositórios)

Você pode criar um `.gitignore` que vale para todos os seus projetos:

```
git config --global core.excludesfile ~/.gitignore_global
```

Exemplo de `.gitignore_global`:

```
# Arquivos do sistema  
Thumbs.db  
.DS_Store  
  
# Configurações de editor  
.vscode/  
.idea/
```



7.6. Dicas profissionais

- Sempre ignore arquivos sensíveis (`.env`, chaves, tokens).
- Nunca coloque senhas no GitHub.

- Evite ignorar arquivos importantes por engano.
- Use `.gitignore` para manter o repositório limpo e leve.

Você pode criar seu `.gitignore` de duas formas:

Copiar um modelo pronto em:

<https://github.com/github/gitignore>

Gerar um `.gitignore` personalizado no [gitignore.io](https://www.toptal.com/developers/gitignore):

<https://www.toptal.com/developers/gitignore>

8. Ver histórico — `git log`

O comando `git log` exibe o histórico de commits do repositório, incluindo:

- hash do commit
- autor
- data
- mensagem do commit

```
git log
```

Exemplo:

```
commit 1a2b3c4
Author: Anderson
Date: Tue Jan 1

    Adiciona script inicial
```

8.1. Versão resumida

Mostra cada commit em uma única linha:

```
git log --oneline
```

Saída típica:

```
1a2b3c4 Adiciona script inicial
```

✂ 8.2. Navegação dentro do log

O git log usa um pager (como o less) para facilitar a navegação.

Um pager é apenas um visualizador que permite rolar textos longos no terminal sem que eles “estourem” a tela. O less é o pager padrão: ele deixa você subir, descer e sair quando quiser, tornando a leitura do histórico muito mais controlada.

Comandos úteis:

- ↑ / ↓ → rolar
- space → próxima página
- b → página anterior
- q → sair

✂ 8.3. Mostrar gráfico das branches

Visualização muito útil para entender merges, branches e a estrutura do histórico:

```
git log --oneline --graph --decorate --all
```

Exemplo:

```
* 1a2b3c4 (HEAD -> main) Adiciona script inicial  
* 9f8e7d6 (feature/login) Cria tela de login
```

✂ 8.4. Limitar quantidade de commits

Mostrar apenas os últimos 5 commits:

```
git log -5
```

✂ 8.5. Filtrar por autor

```
git log --author="Anderson"
```

✂ 8.6. Filtrar por palavra na mensagem

```
git log --grep="login"
```

✂ 8.7. Mostrar alterações junto com o commit

```
git log -p
```

Mostra o patch (as diferenças) de cada commit.

✦ 8.8. Mostrar apenas nomes de arquivos alterados

```
git log --name-only
```

✦ 8.9. Mostrar estatísticas resumidas

```
git log --stat
```

Exemplo:

```
script.py | 10 ++++++++  
1 file changed, 10 insertions(+)
```

✦ 8.10. Combinação útil para o dia a dia

```
git log --oneline --decorate --graph --all
```

Essa visualização é muito usada por desenvolvedores porque mostra:

- commits resumidos
- branches
- merges
- posição do HEAD
- tags
- estrutura visual do histórico



9. Fluxo básico de trabalho (local)

```
git init  
git config --global user.name "Nome"  
git config --global user.email "Email"  
git status  
git add .  
# Obs.: é necessário criar ou modificar um arquivo para enviá-lo ao staging area
```

```
git commit -m "mensagem"  
git log --oneline
```

Fim do Guia 1 — Git Básico + Configurações

Agora você entende:

- como iniciar um repositório
- como configurar o Git
- como interpretar saídas do terminal
- como adicionar arquivos
- como fazer commits
- como ignorar arquivos
- como ver o histórico
- como seguir o fluxo básico de trabalho

Esse é o alicerce para todos os próximos guias.
